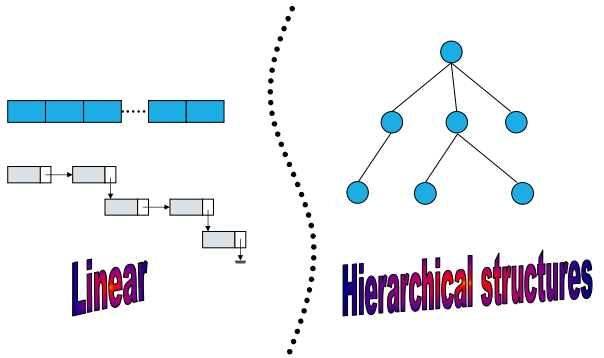
Nội dung

- Cấu trúc cây
- Cây nhị phân
- Cây nhị phân tìm kiếm
- Duyệt cây
- Cây AVL
- Cây đỏ đen
- Cây B-Tree

3

Cấu trúc dữ liệu



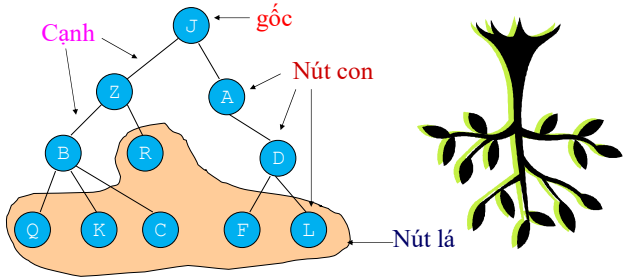
5

Cấu trúc cây

- Tập hợp các nút và cạnh nối các nút đó.
- Có một nút gọi là *gốc*.
- Quan hệ one-to-many giữa các nút.
- Có duy nhất một đường đi từ gốc đến nút con.
- Các loại cây:
  - **Nhị phân**: mỗi nút có {0, 1, 2} nút con
  - **Tam phân**: mỗi nút có {0, 1, 2, 3} nút con
  - **n-phân**: mỗi nút có {0, 1, ..., n} nút con

6

Khái niệm

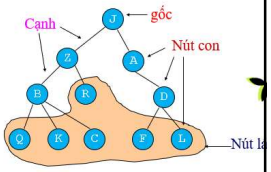


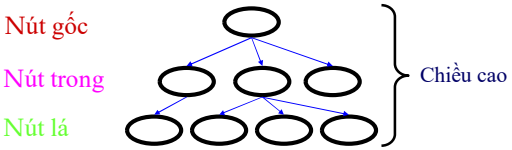
8

Khái niệm

Thuật ngữ

- **Nút gốc (root)**: không có nút cha
- **Nút lá (leaf)**: không có nút con
- **Nút trong, nút trung gian (interior node)**: không phải nút lá và nút gốc
- **Nút hậu duệ (descendant) & nút tiền bối (ancestor)**: Nếu có đường đi từ nút a đến nút b thì nút a là tiền bối của b, còn b là hậu duệ của a.
- **Chiều cao**: khoảng cách từ gốc đến lá của nhánh cao nhất

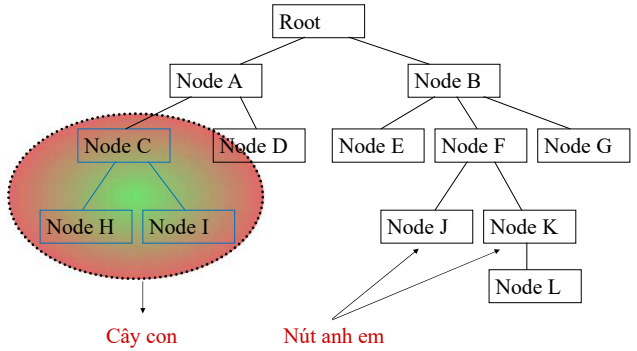




9

Khái niệm

Cây con của 1 cây là 1 nút cùng với tất cả các hậu duệ của nó.



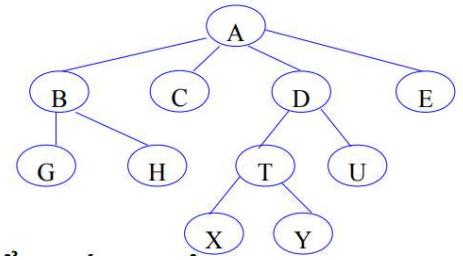
10

Duyệt cây

- Duyệt cây:
- – Quy tắc: đi qua lần lượt tất cả các nút của cây, mỗi nút đúng một lần
- – Danh sách duyệt cây: là danh sách liệt kê các nút theo thứ tự đi qua
- – Có 3 phương pháp duyệt tổng quát:
  - tiền tự (preorder) = NLR
  - trung tự (inorder) = LNR
  - hậu tự (posorder) = LRN

11

Duyệt cây



NLR ?

LNR?

LRN ?

12

Cây nhị phân

- Cây nhị phân là một đồ thị có hướng, mỗi nút có tối đa 2 nút con, *hay Cây nhị phân là cây có bậc bằng hai (bậc của mỗi nút tối đa bằng 2)*.
- Cấu trúc cây đơn giản nhất
- Tại mỗi nút gồm các 3 thành phần
  - Phần **Data**: chứa giá trị, thông tin của nút
  - Phần **Left**: Liên kết đến nút con trái (nếu có)
  - Phần **Right**: Liên kết đến nút con phải (nếu có)



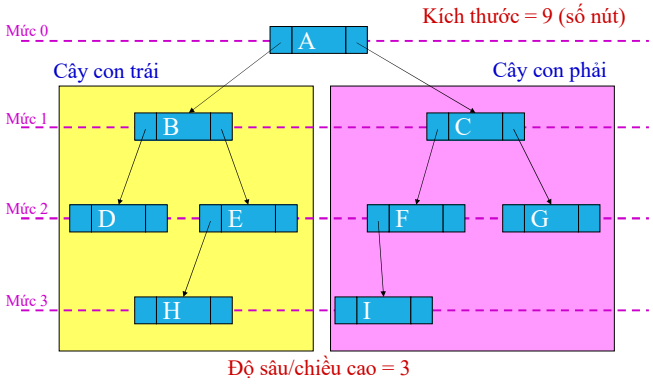
15

Cây nhị phân

- Cây nhị phân có thể rỗng (không có nút nào)
- Cây nhị phân khác rỗng có 1 nút gốc
  - Có duy nhất 1 đường đi từ gốc đến 1 nút con.
  - Nút không có nút con bên trái và con bên phải là nút lá.

16

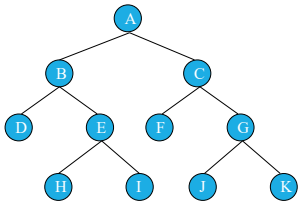
Cây nhị phân



17

Cây nhị phân

- Cây nhị phân đúng:
  - Nút gốc và nút trung gian có đúng 2 con
- Cây nhị phân đúng có n nút lá thì số nút trên cây 2* n – 1.



18

Cấu trúc cây nhị phân

- Cấu trúc của cây nhị phân.

```
class TNode
{    //Cấu trúc của một nút
     ItemType data;
     TNode  left;
     TNode  right;
}
class BTree
{    //Cấu trúc của một cây
     TNode root;
}
```

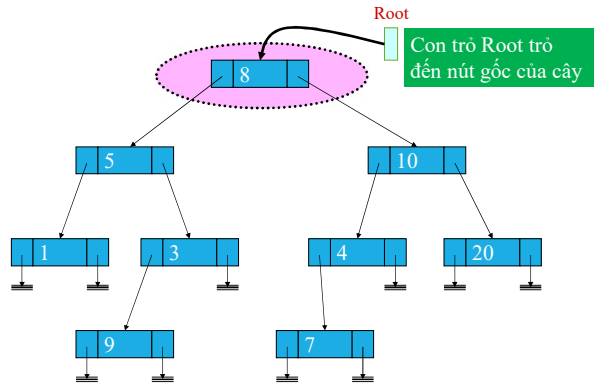
Chứa thông tin của nút

Trỏ đến nút con trái

Trỏ đến nút con phải

Con trỏ đến nút gốc của cây

Sơ đồ cây nhị phân



Các thao tác cơ bản

- Khởi tạo cây
- Tạo nút mới.
- Thêm nút con trái T
- Thêm nút con phải T
- Thêm nút có giá trị x
- Duyệt cây
- Xóa con trái T
- Xóa con phải T
- Xóa nút có giá trị x
- Xóa cây.

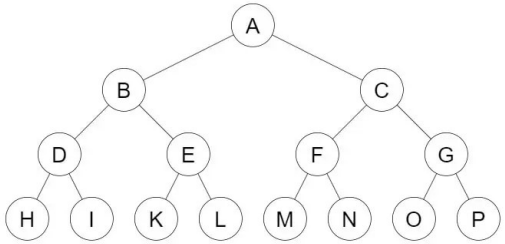
Duyệt cây

- Do cây là cấu trúc không tuyến tính
- Có 6 cách duyệt cây nhị phân:
  - Duyệt theo thứ tự trước **PreOrder**: **traverseNLR**, **traverseNRL**
  - Duyệt theo thứ tự giữa **InOrder**: **traverseLNR**, **traverseRNL**
  - Duyệt theo thứ tự sau **PostOrder**: **traverseLRN**, **traverseRLN**

Duyệt cây

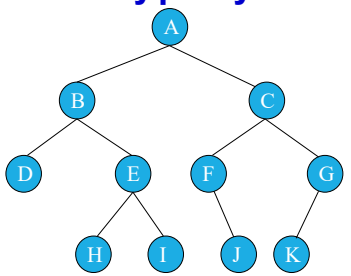
Duyệt cây theo các phương pháp

1. NLR
2. LNR
3. LRN



29

Duyệt cây



PreOrder (traverseNLR)

InOrder (traverseLNR)

PostOrder (traverseLRN)

31

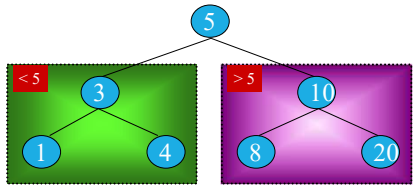
Nội dung

- Cấu trúc cây
- Cây nhị phân
- **Cây nhị phân tìm kiếm**
- Duyệt cây
- Cây AVL
- Cây đỏ đen
- Cây B-Tree

55

Khái niệm

- BST (Binary Search Tree) là cây nhị phân mà mỗi nút thoả:
  - Giá trị của tất cả nút con trái < nút gốc
  - Giá trị của tất cả nút con phải > nút gốc



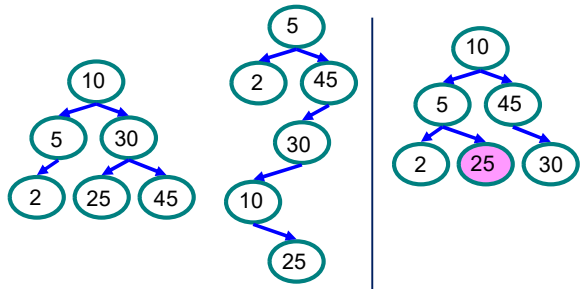
57

Cây nhị phân tìm kiếm

- Xây dựng cây BST
  - Tìm kiếm/ thêm
  - Xóa
- Luôn duy trì tính chất
  - Giá trị nhỏ hơn ở bên cây con trái
  - Giá trị lớn hơn ở bên cây con phải

59

Ví dụ

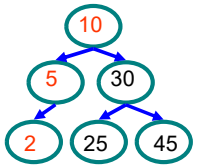


Binary search trees

Non-binary search tree

61

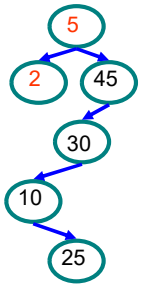
Ví dụ tìm x = 2



10 > 2, Left

5 > 2, Left

2 = 2, Tìm thấy

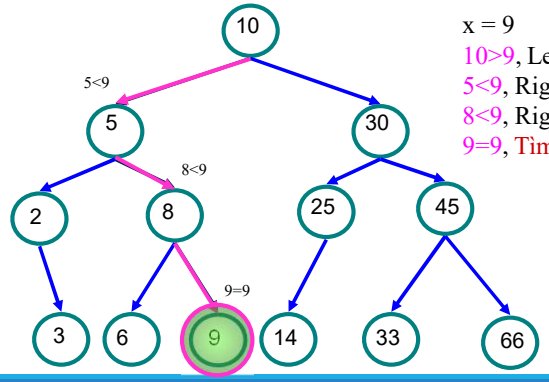


5 > 2, Left

2 = 2, Tìm thấy

62

Ví dụ tìm x= 9



x = 9

10>9, Left

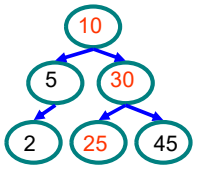
5<9, Right

8<9, Right

9=9, Tìm thấy

63

Ví dụ tìm x = 25



10 < 25, Right

30 > 25, Left

25 = 25, Tìm thấy

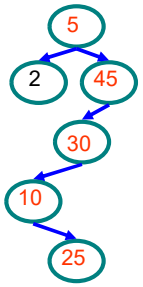
5 < 25, Right

45 > 25, Left

30 > 25, Left

10 < 25, Right

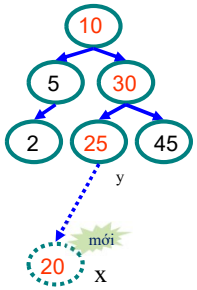
25 = 25, Tìm thấy



64

Thêm x vào cây

- Thực hiện tìm kiếm giá trị x
- Tìm đến cuối nút y(nếu x không tồn tại trong cây)
- Nếu x < y, thêm nút lá x bên trái của y
- Nếu x > y, thêm nút lá x bên phải của y



70

Xóa phần tử khỏi cây

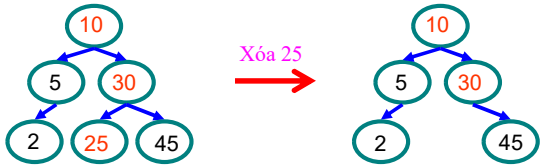
Xóa nhưng phải đảm bảo vẫn là cây BST

- Thực hiện tìm nút có giá trị x.
- Nếu nút là nút lá, xóa nút.
- Ngược lại
  - Thay thế nút bằng một trong hai nút sau.
    - Y là nút lớn nhất của cây con bên trái
    - Z là nút nhỏ nhất của cây con bên phải
  - Chọn nút Y hoặc Z để thế chỗ.
  - Giải phóng nút có giá trị x.

72

Ví dụ xóa x = 25

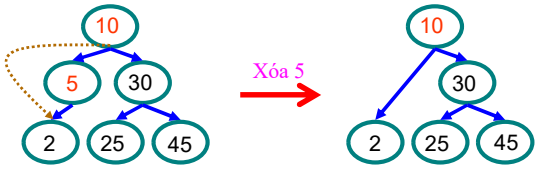
- Trường hợp 1: nút p là nút lá, xóa bình thường



73

Ví dụ xóa x = 5

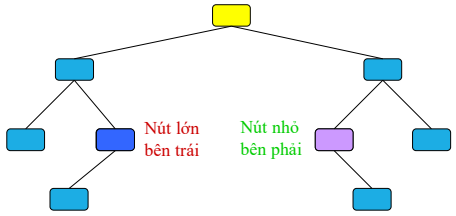
- Trường hợp 2: p chỉ có 1 cây con, cho nút cha của p trỏ tới nút con duy nhất của nó, rồi hủy p



74

Ví dụ

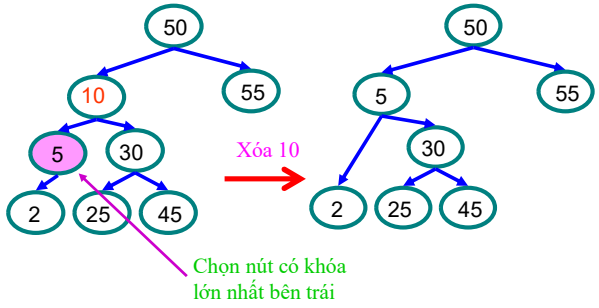
- Trường hợp 3: nút p có 2 cây con, chọn nút thay thế theo 1 trong 2 cách như sau:
  - Nút lớn nhất trong cây con bên trái
  - Nút nhỏ nhất trong cây con bên phải



75

Ví dụ

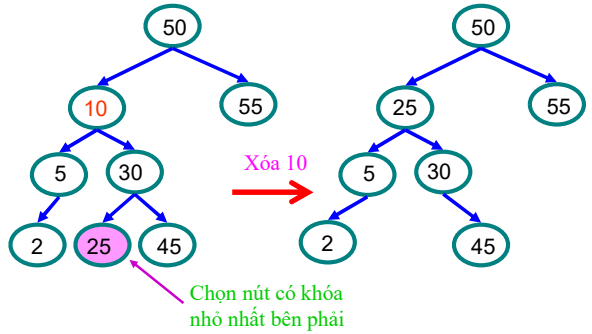
- Xóa nút 10: cách 1



76

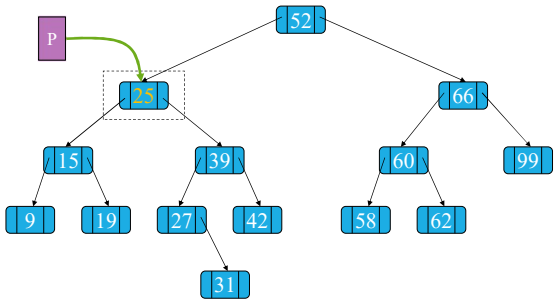
Ví dụ

- Xóa nút 10: cách 2



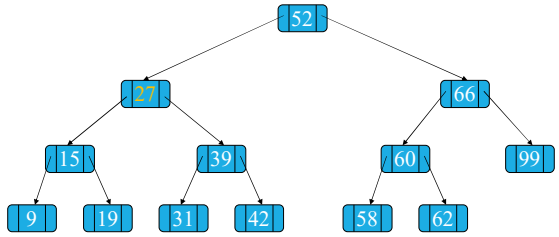
77

Ví dụ xóa x = 25



78

Ví dụ xóa x = 27



83

Nội dung

- Cấu trúc cây
- Cây nhị phân
- Cây nhị phân tìm kiếm
- **Duyệt cây**
- Cây AVL
- Cây đỏ đen
- Cây B-Tree

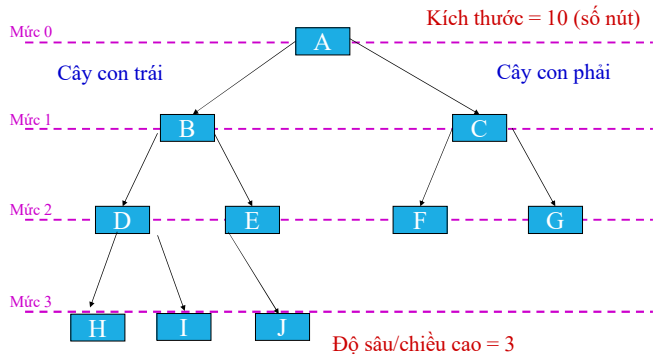
95

Duyệt cây theo chiều rộng

- Duyệt cây theo chiều rộng là cách duyệt cây theo một mức hay độ sâu nhất định trước khi duyệt tới mức tiếp theo sâu hơn.

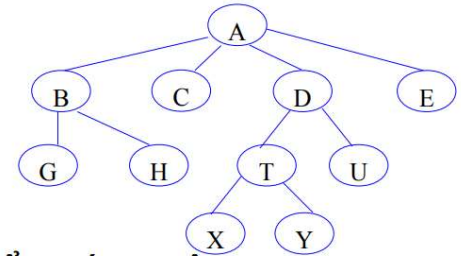
96

Cây nhị phân



97

Duyệt cây theo BFS



99

Duyệt cây theo chiều sâu

❖Có 3 cách duyệt theo chiều sâu cây nhị phân là: Preorder, Inorder, Postorder.

Cách duyệt	Thứ tự		
Preorder	<gốc>	<cây con trái>	<cây con phải>
Inorder	<cây con trái>	<gốc>	<cây con phải>
Postorder	<cây con trái>	<cây con phải>	<gốc>

122

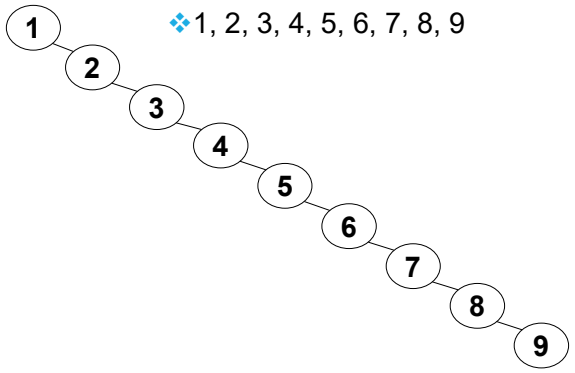
Nội dung

- Cấu trúc cây
- Cây nhị phân
- Cây nhị phân tìm kiếm
- Duyệt cây
- **Cây AVL**
- Cây đỏ đen
- Cây B-Tree

130

Đánh giá tìm kiếm

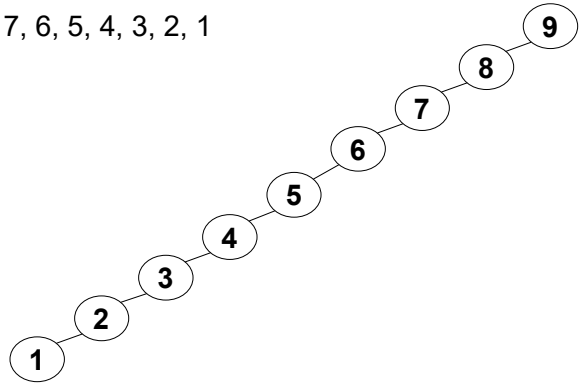
- 1, 2, 3, 4, 5, 6, 7, 8, 9



131

Đánh giá tìm kiếm

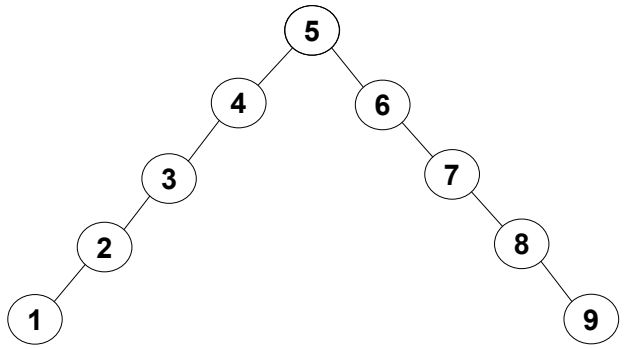
- 9, 8, 7, 6, 5, 4, 3, 2, 1



132

Đánh giá tìm kiếm

- 5, 4, 3, 2, 1, 6, 7, 8, 9



133

AVL Tree - Giới thiệu

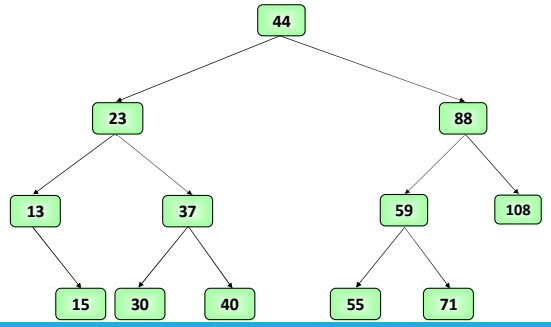
- Phương pháp thêm trên Cây NPTK có thể có những biến dạng mất cân đối nghiêm trọng
  - **Chi phí cho việc tìm kiếm** trong trường hợp xấu nhất đạt tới **n**
  - VD: 1 triệu nút ⇒ chi phí tìm kiếm = 1.000.000 nút
- Nếu có một cây tìm kiếm nhị phân cân bằng hoàn toàn, **chi phí cho việc tìm kiếm** chỉ xấp xỉ $\log_2 n$
  - VD: 1 triệu nút ⇒ chi phí tìm kiếm = $\log_2 1.000.000 \approx 20$ nút
- G.M. Adelson - Velsky và E.M. Landis đã đề xuất một tiêu chuẩn cân bằng (gọi là cân bằng **AVL**)
  - Cây AVL có chiều cao $O(\log_2(n))$

134

AVL Tree - Định nghĩa

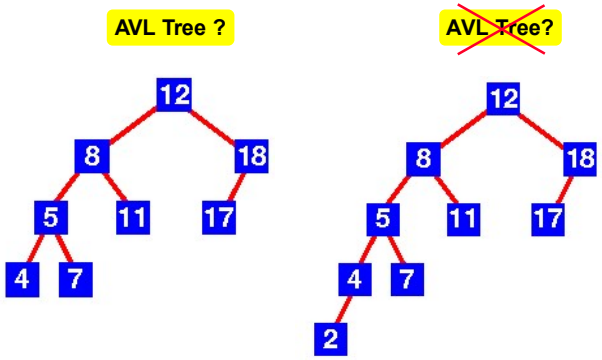
- Cây nhị phân tìm kiếm cân bằng là cây mà **tại mỗi nút** của nó **độ cao** của cây con trái và của cây con phải **chênh lệch nhau không quá một**.

Ví dụ:



136

AVL Tree – Ví dụ



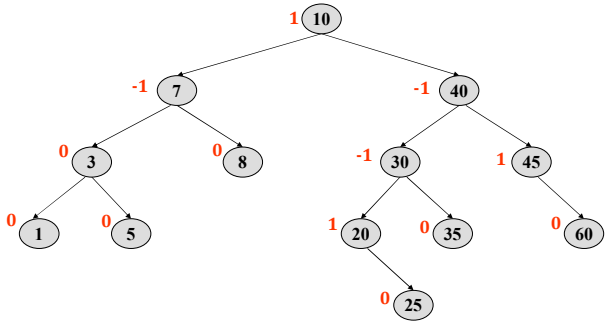
137

Tổ chức dữ liệu

- **Chỉ số cân bằng của một nút:**
  - **Định nghĩa**: Chỉ số cân bằng của một nút là **hiệu** của chiều cao cây con phải và cây con trái của nó.
  - Đối với một cây cân bằng, chỉ số cân bằng (CSCB) của mỗi nút chỉ có thể mang một trong ba giá trị: -1, 0, và 1:
- **Các giá trị hợp lệ:**
  - **CSCB(p)=0** ⇔ Độ cao cây trái (p) **=** Độ cao cây phải (p)
  - **CSCB(p)=1** ⇔ Độ cao cây trái (p) **<** Độ cao cây phải (p)
  - **CSCB(p)=-1** ⇔ Độ cao cây trái (p) **>** Độ cao cây phải (p)

138

Ví dụ - Chỉ số cân bằng của nút



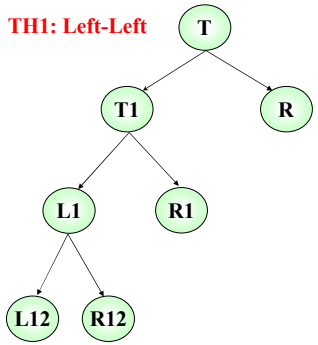
- Hãy cho biết chỉ số cân bằng của mỗi nút trên cây?
- Đây có phải là cây AVL không?

139

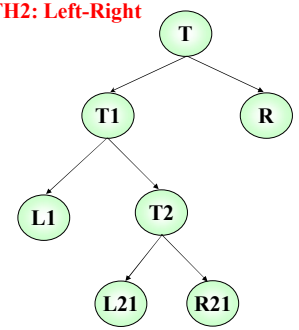
Các trường hợp mất cân bằng do lệch trái

➢Cây mất cân bằng tại nút T

TH1: Left-Left



TH2: Left-Right

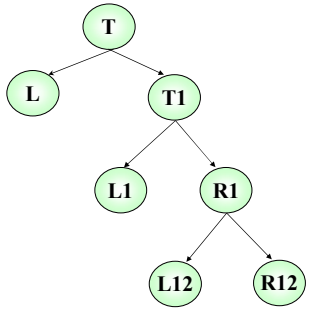


146

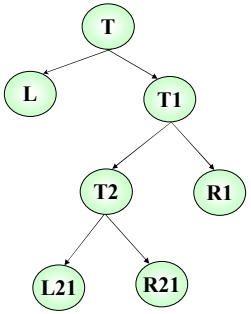
Các trường hợp mất cân bằng do lệch phải

➢Cây mất cân bằng tại nút T

TH3: Right-Right



TH4: Right-Left

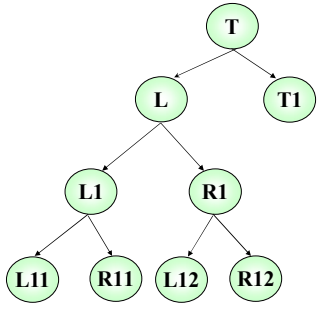


147

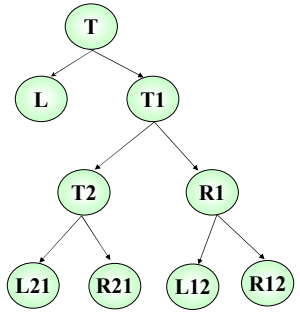
Các trường hợp mất cân bằng do xóa

➢Cây mất cân bằng tại nút T

TH5: Left-Balance



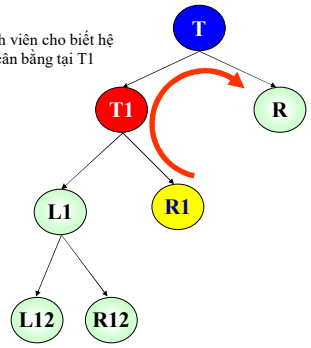
TH6: Right-Balance

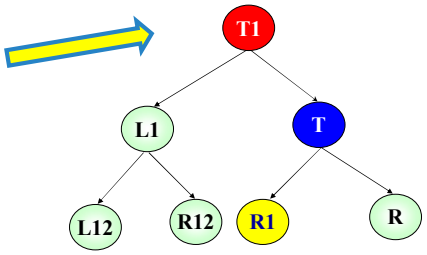


148

Cân bằng lại trường hợp 1: Left-Left

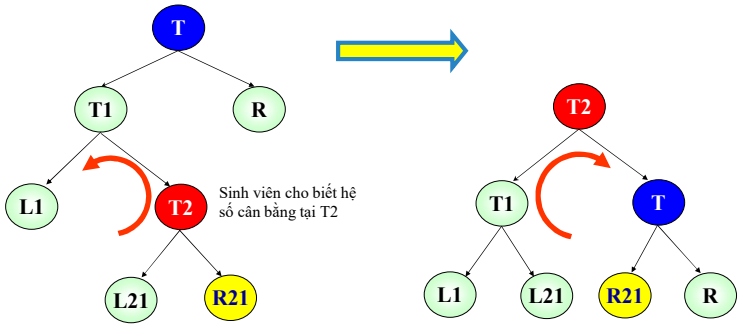
Sinh viên cho biết hệ số cân bằng tại T1





149

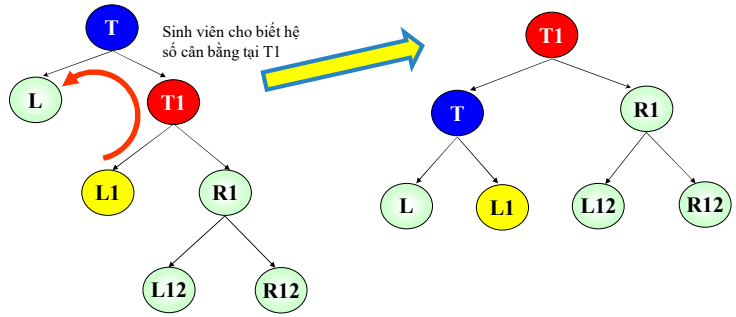
Cân bằng lại trường hợp 2: Left-Right



Xoay trái tại nút T_1 Xoay phải tại nút T_2

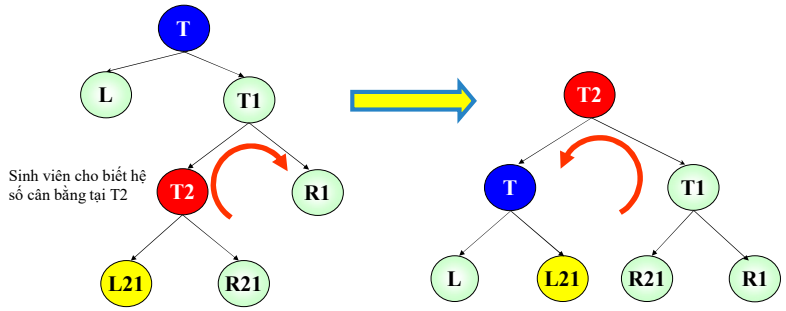
151

Cân bằng lại trường hợp 3: Right - Right



153

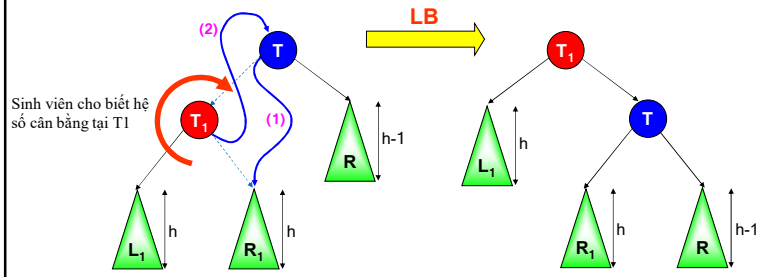
Cân bằng lại trường hợp 4: Right - Left



Xoay phải tại nút T_1 Xoay trái tại nút T_2

155

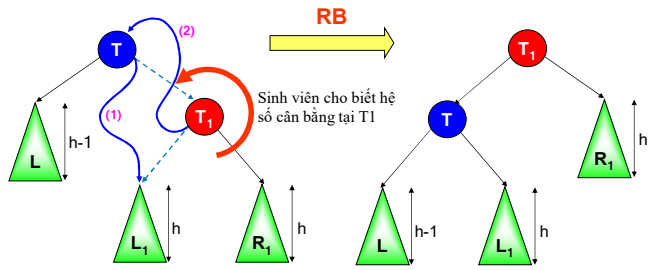
Cân bằng lại trường hợp 5: Left - Balance



$T \rightarrow Left = T_1 \rightarrow Right$; (1)
$T_1 \rightarrow Right = T$; (2)

158

Cân bằng lại trường hợp 6: Right - Balance



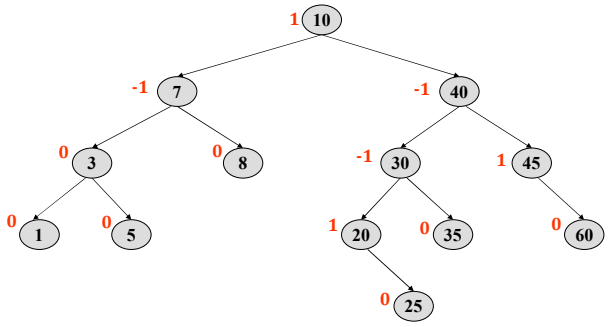
$T \rightarrow Right = T_1 \rightarrow Left;$ (1)
$T_1 \rightarrow Left = T;$ (2)

161

Luyện tập

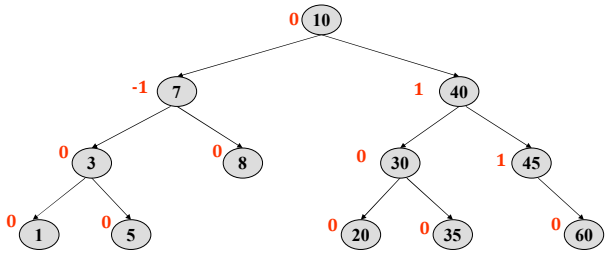
168

Ví dụ cho cây cân bằng AVL, hãy xóa node 25 (1)



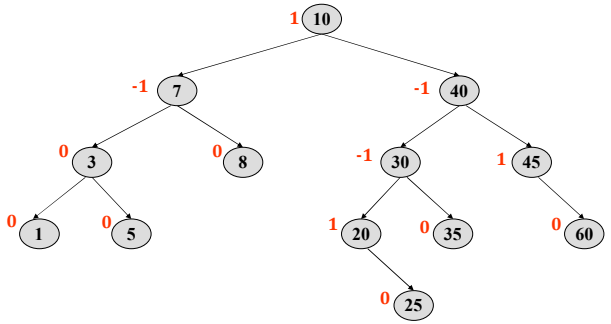
169

Ví dụ xóa node 25 (2)



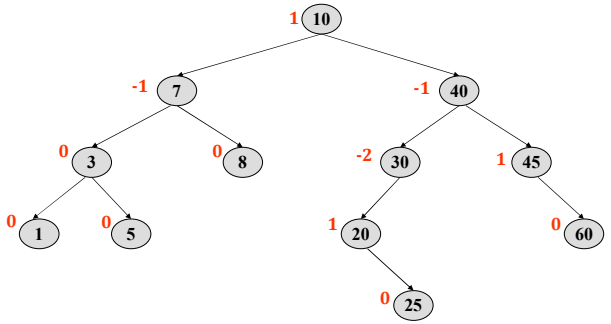
170

Ví dụ xóa node 35



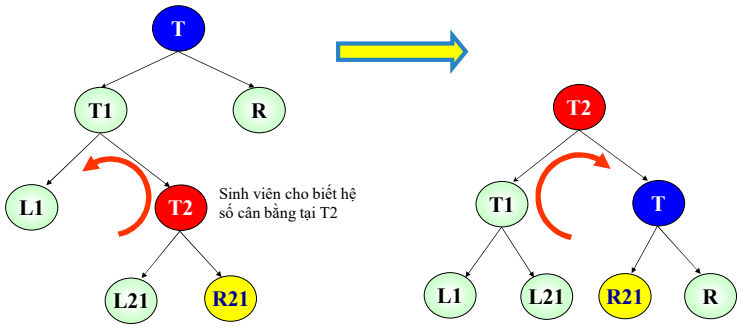
171

Ví dụ: Xóa node 35 (1)



172

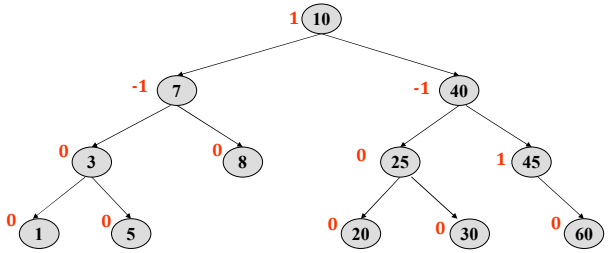
Cân bằng lại trường hợp 2 (L-R)



Xoay trái tại nút T_1 Xoay phải tại nút T_2

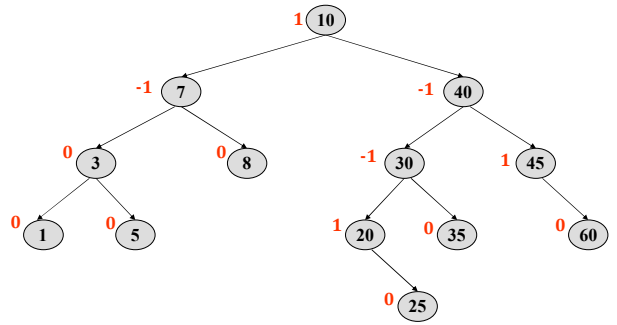
173

Ví dụ: Xóa node 35 (2)



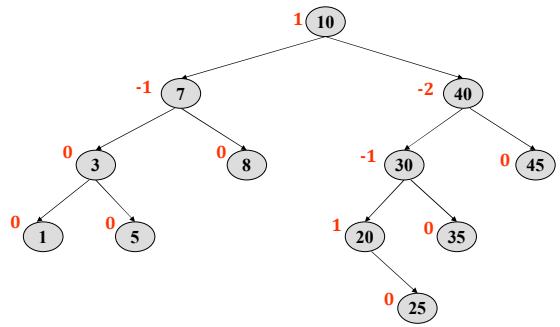
174

Ví dụ: Xóa node 60



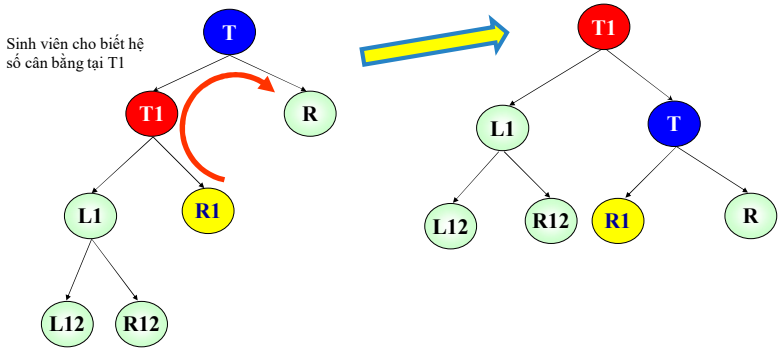
175

Ví dụ: Xóa node 60 (1)



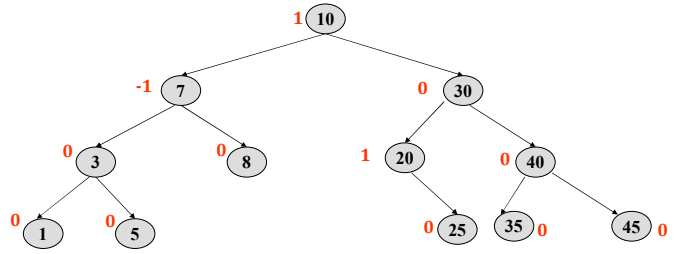
176

Cân bằng lại trường hợp 1 (L-L)



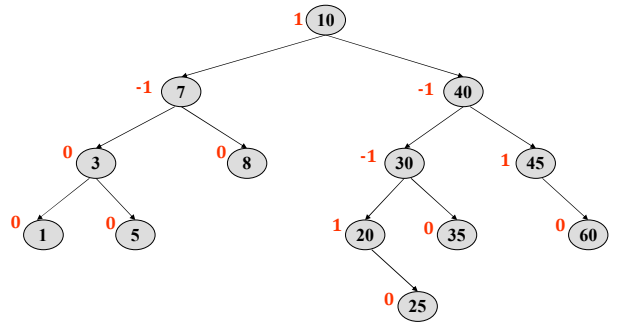
177

Ví dụ: Xóa node 60 (2)



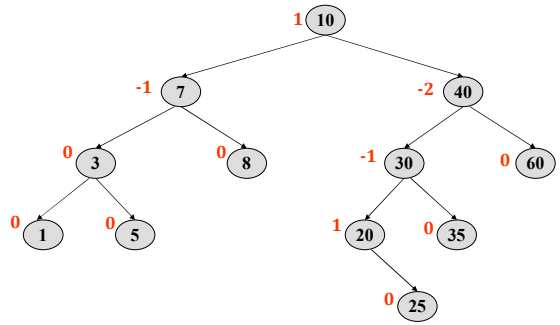
178

Ví dụ: Xóa node 45



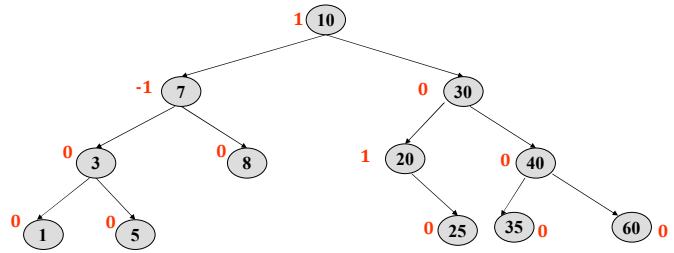
179

Ví dụ: Xóa node 45 (1)



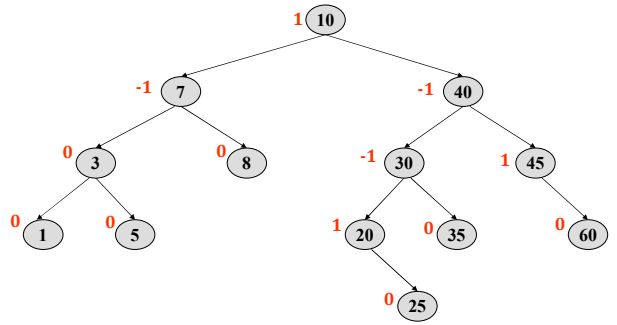
180

Ví dụ: Xóa node 45 (2)



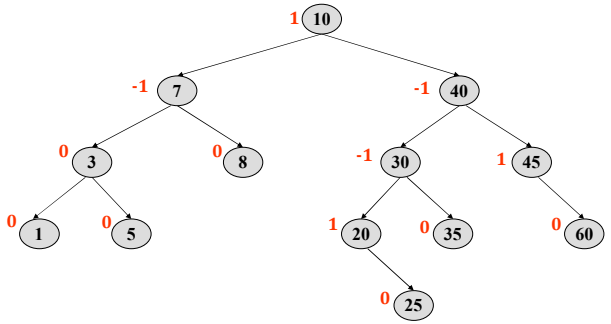
181

Ví dụ: Xóa node 30



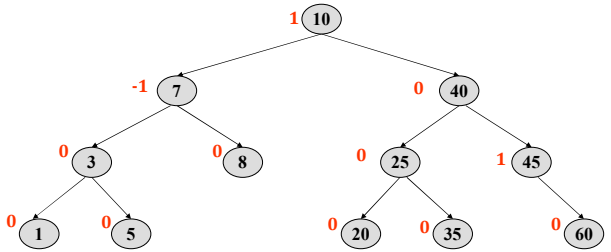
182

Ví dụ: Xóa node 30 (1)



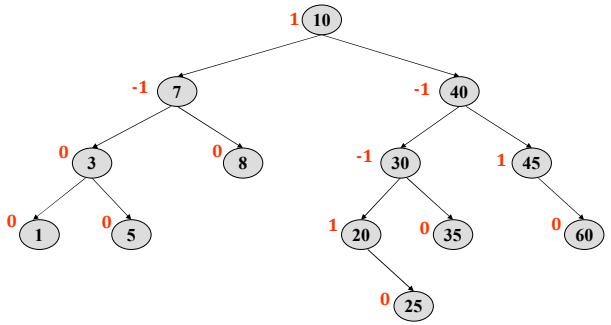
183

Ví dụ: Xóa node 30 (2)



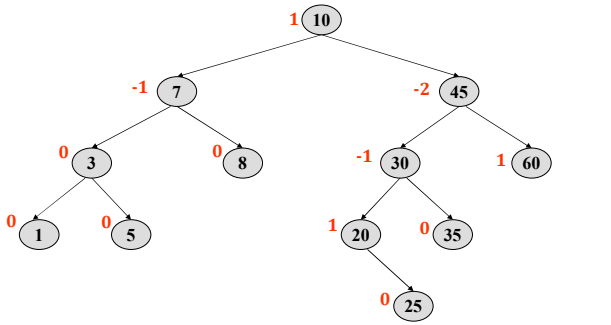
184

Ví dụ: Xóa node 40



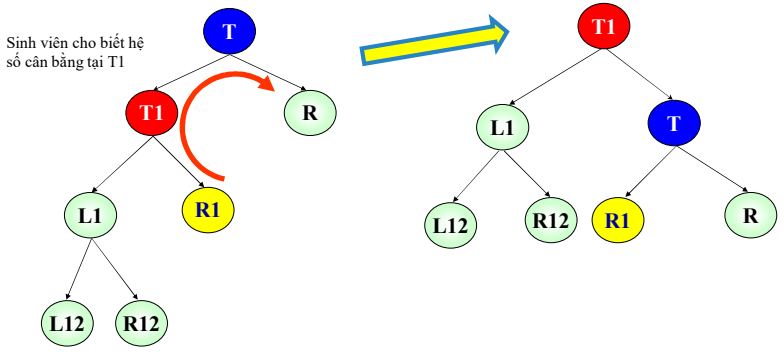
185

Ví dụ: Xóa node 40 (1)



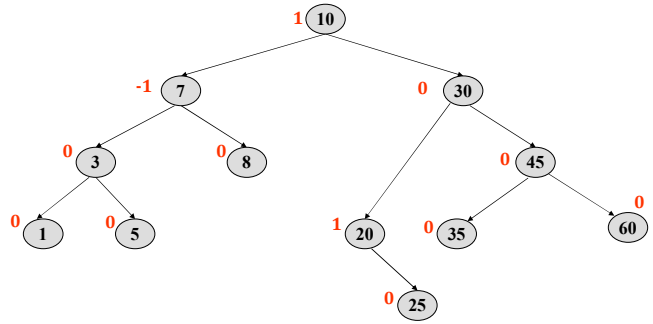
186

Cân bằng lại trường hợp 1 (L-L)



187

Ví dụ: Xóa node 40 (2)



188

Nội dung

- Cấu trúc cây
- Cây nhị phân
- Cây nhị phân tìm kiếm
- Duyệt cây
- Cây AVL
- **Cây đỏ đen**
- Cây B-Tree

189

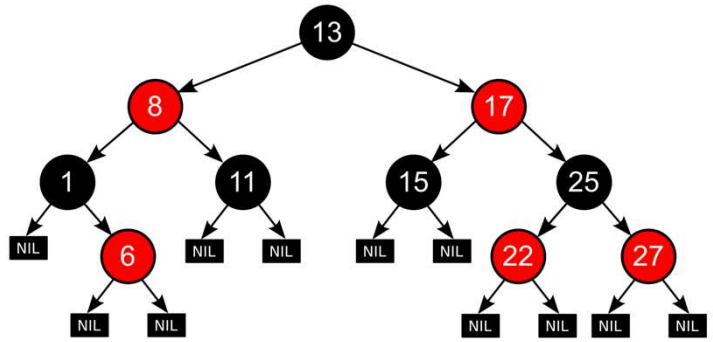
Định nghĩa cây đỏ đen

Cây đỏ-đen là một nhị phân tìm kiếm (BST) thỏa các điều kiện sau:

1. Mọi node đều được tô màu đỏ hoặc màu đen.
2. Node gốc luôn đen.
3. Node lá (NIL) luôn đen.
4. Cả hai con của mọi nút đỏ là đen. (và suy ra mọi nút đỏ có nút cha là đen)
5. Tất cả các đường đi từ một nút bất kỳ tới các lá có số nút đen bằng nhau. Số các nút đen trên một đường đi từ gốc tới mỗi lá được gọi là độ dài đen của đường đi đó.

191

Cây đỏ đen



192

Chèn node

- Phép chèn bắt đầu bổ sung một nút như BST và gán cho nó màu đỏ. Bảo toàn tính chất đỏ đen từ các nút lân cận với nút mới bổ sung.
- Thuật ngữ *nút chú bác* sẽ dùng để chỉ nút anh (hoặc em) với nút cha của nút đó như trong cây phả hệ.
- **N** sẽ dùng để chỉ nút đang chèn vào, **P** chỉ nút cha của **N**, **G** chỉ ông của **N**, và **U** chỉ chú bác của **N**.

193

Chèn node

TH1: Chèn **N** tại gốc. Trong trường hợp này, gán lại màu đen cho **N**, để bảo bảo tính chất gốc là đen. Vì chỉ bổ sung một nút, Tính chất 4 được bảo đảm vì mọi đường đi chỉ có một nút.

195

Chèn node

TH2: Nút cha **P** của nút mới thêm là đen, khi đó Tính chất 4 (Cả hai nút con của nút đỏ là đen) không bị vi phạm vì nút mới thêm có hai con là "null' là đen. Tính chất 5 cũng không vi phạm vì nút mới thêm là đỏ không ảnh hưởng tới số nút đen trên tất cả đường đi.

197

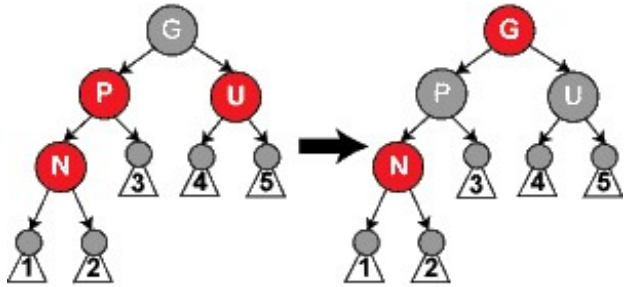
Chèn node

❖**TH3:** Cả cha **P** và bác **U** là đỏ, thì đổi cả hai thành đen còn **G** thành đỏ (để bảo toàn tính chất 5) .Khi đó nút mới **N** có cha đen. Vì đường đi bất kỳ đi qua cha và bác của "N" phải đi qua ông của *N* nên số các nút đen trên đường đi này không thay đổi.

❖Để sửa chữa trường hợp này gọi một thủ tục đệ quy trên **G** từ trường hợp 1.

199

Chèn node



200

Chèn node

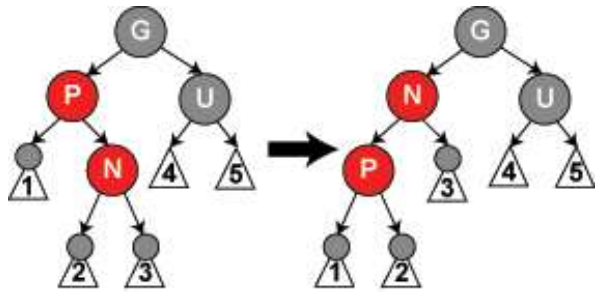
❖**TH4:** Nút cha **P** là đỏ nhưng nút chú bác **U** là đen, nút mới **N** là con phải của nút **P**, và **P** là con trái của nút **G**.

❖Thực hiện quay trái chuyển đổi vai trò của nút mới **N** và nút cha **P** do đó định dạng lại nút **P** bằng Trường hợp 5 (đổi vai trò **N** và **P**) vì tính chất 4 bị vi phạm (Cả hai con của nút đỏ là đen).

❖Phép quay cũng làm thay đổi một vài đường đi (các đường đi qua cây con nhãn "1") phải đi qua thêm nút mới **N**, nhưng vì **N** là đỏ nên không làm chúng vi pham tính chất 5

202

Chèn node



203

Chèn node

❖**TH5:** Nút cha **P** là đỏ nhưng nút bác **U** là đen, nút mới **N** là con trái của nút **P**, và **P** là con trái của nút ông **G**.

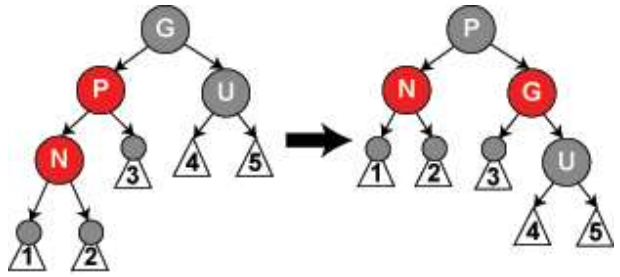
❖Phép quay phải trên nút ông **G** được thực hiện; kết quả của phep quay là trong cây mới nút **P** trở thành cha của cả hai nút **N** và nút **G**. **G** là đen, vì bây giờ nó là con của **P**.

❖Đổi màu của **P** và **G** thì cây thỏa mãn tính chất 4.

❖Tính chất 5 không bị vi phạm vì các đường đi qua G trước đây bây giờ đi qua **P**.

205

Chèn node



206

Xóa node

❖**TH1:** N là gốc mới. Ta dừng lại. Ta đã giải phóng một nút đen khỏi mọi đường đi và gốc mới lại là đen. Không tính chất nào bị vi phạm.

```
void delete_case1(struct node *n) {
    if (n->parent == NULL)
        return;
    else
        delete_case2(n);
}
```


210

Xóa node

❖**TH2:** **S** là đỏ. Tráo đổi màu của **P** và **S**, và sau đó quay trái tai **P**, **S** trở thành nút ông của **N**.

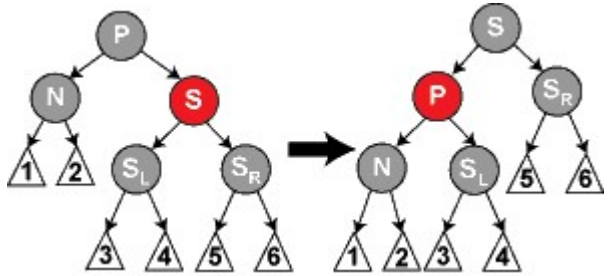
❖**P** có màu đen và có một con màu đỏ. Tất cả các đường đi có số các nút đen giống nhau

❖**N** có một anh em màu đen và cha màu đỏ, chúng ta có thể tiếp tục với các trường hợp 4, 5, hoặc 6. (anh em mới của nó là đen ví chỉ có một con của nút đỏ **S**.)

❖Trong các trường hợp sau ta sẽ gọi anh em mới của **N'** là **S**.

211

Xóa node



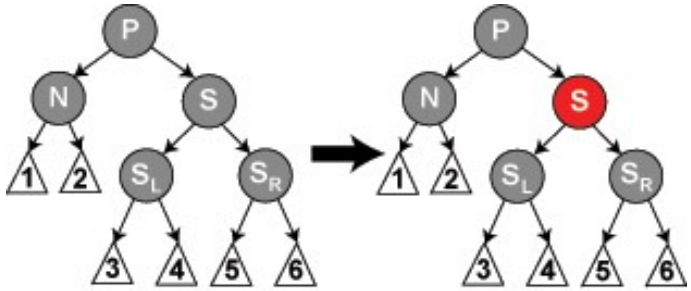
212

Xóa node

- **TH3:** **P**, **S**, và các con của **S** là đen. Gán lại cho **S** màu đỏ. Kết quả là mọi đường đi qua **S**, (tất nhiên chúng không qua **N**,có ít hơn một nút đen.
- Vì việc xóa đi cha trước đây của **N'** làm tất cả các đương đi qua **N** bớt đi một nút đen, nên chúng bằng nhau.
- Tuy nhiên tất cả các đường đi qua **P** bây giờ có ít hơn một nút đen so với các đường không qua **P**, do đó Tính chất 5 (Tất cả các đường đi từ gốc tới các nút lá có cùng số nút đen) sẽ bị vi phạm. Để sửa chữa tái cân bằng tại **P**, bắt đầu từ trường hợp 1

214

Xóa node



215

Nội dung

- Cấu trúc cây
- Cây nhị phân
- Cây nhị phân tìm kiếm
- Duyệt cây
- Cây AVL
- Cây đỏ đen
- **Cây B-Tree**

217

NỘI DUNG

1. Giới thiệu
2. Khái niệm
3. Đặc điểm và cấu trúc
4. Chèn phần tử vào cây
5. Xóa phần tử khỏi cây

GIỚI THIỆU

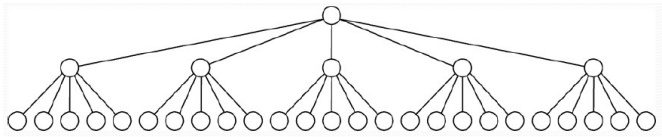
- Cây là một cách tiếp cận hoàn chỉnh để tổ chức dữ liệu trong bộ nhớ. Vậy cây có thể làm việc tốt với hệ thống tập tin hay không?
- B-Tree là cấu trúc dữ liệu phù hợp cho việc lưu trữ ngoài do R.Bayer và E. M. McCreight đưa ra năm 1972.

GIỚI THIỆU

- B-Tree là cây tìm kiếm tự cân bằng. Trong hầu hết các cây tìm kiếm tự cân bằng khác (như AVL và Red Black Trees), giả định rằng mọi thứ đều nằm trong bộ nhớ chính.
- Để hiểu được việc sử dụng B-Tree, chúng ta phải nghĩ đến số lượng dữ liệu khổng lồ mà không thể lưu trong bộ nhớ chính. Khi số lượng keys lớn, dữ liệu được đọc từ hard disk dưới dạng khối (blocks). Thời gian đọc dữ liệu từ hard disk rất lâu so với thời gian truy xuất bộ nhớ chính → sử dụng B-Tree là giảm số lần truy cập đĩa.
- Hầu hết các hoạt động của cây (tìm kiếm, chèn, xóa, max, min, ..etc) yêu cầu truy cập đĩa O(h) chính là chiều cao của B-Tree.
- Chiều cao của B-Tree được hạn chế tối đa bằng việc bố trí nhiều nhất keys tại các node (kích thước node tương đương kích thước block). Vì khả năng hạn chế h, tổng số lần đĩa truy cập cho hầu hết các hoạt động được giảm đáng kể so với cây AVL, Red Black Tree, ..etc.

Cây nhiều nhánh: M-Phân

- Mỗi node có tối đa M node con (*cây con*)
- Một cây M-Phân đầy đủ có chiều cao $log_M N$
- Ví dụ: Cây 5-Phân đầy đủ như sau



Định nghĩa

- Một B-Tree bậc M là cây M-Phân là cây thỏa:
  - Mỗi node có tối đa **M** cây con, và có **M-1** khóa.
  - Các khóa trong mỗi node (cây con) được sắp xếp tăng.
  - Các khóa trong cây con thứ i đều nhỏ hơn khóa i.
  - Các khóa trong cây con thứ (i+1) đều lớn hơn khóa i.

Tính chất

- Cây M-Phân tìm kiếm có các tính chất sau:
  - Tất cả node lá có cùng mức.
  - Tất cả các node trung gian (***trừ node gốc***) có nhiều nhất M cây con và có ít nhất M/2 cây con (khác rỗng).
  - Mỗi node hoặc là node lá hoặc có k+1 cây con (k là số khoá của node này).
  - Node gốc có nhiều nhất M cây con hoặc có thể có 2 cây con (Node gốc có 1 khoá và không phải là node lá) hoặc không chứa cây con nào (Node gốc có 1 khoá và cũng là node lá).

Tính chất

- Cây M-Phân tìm kiếm có các tính chất sau:
  - Tại thời điểm chèn một nút đầy đủ, cây chia thành hai phần và khóa có giá trị trung bình được chèn tại nút cha.
  - Hoạt động hợp nhất diễn ra khi các nút bị xóa.

Ý nghĩa

- B-cây là cây cân bằng hoàn toàn.
- Mỗi node được lấp đầy ít nhất 50%. Các phân tích và thử nghiệm thực tế cho thấy các node của B-cây trong trường hợp bình thường được lấp đầy ~70%
- B-cây sử dụng số phép truy xuất đĩa tối thiểu cho các thao tác.
- Thích hợp với việc lưu trữ trên bộ nhớ ngoài.
- Có thể quản lý số phần tử rất lớn.

Ý nghĩa

- Hạn chế số thao tác đọc mỗi lần tìm kiếm trên cây.
- Thích hợp cho việc tìm kiếm trên bộ nhớ ngoài.
- Chiều cao h của cây ~ $log_M N$, N là số nút của cây.

  Cụ thể: Nếu N là số khoá (N ≥ 1), M là bậc của cây (M > 2) thì: $h \leq log_M \frac{(N+1)}{2}$
- Tăng M chiều cao cây giảm rất nhanh.

Chèn node vào cây

Ý tưởng: Tìm vị trí khóa có thể thêm vào cây. Việc tìm kiếm sẽ kết thúc tại một nút lá. Khóa mới sẽ được thêm vào nút lá:

1. Nếu chưa đầy → Việc thêm hoàn tất.
2. Nếu đầy → Phân đôi nút lá cần thêm:
   - **Tách nút lá** ra làm hai nút cạnh nhau trong cùng một mức.
   - Chuyển phần tử giữa lên nút cha.

Quá trình phân đôi các nút có thể được lan truyền ngược về đến nút gốc và kết thúc khi có một nút cha nào đó cần được thêm một khóa từ dưới lên mà chưa đầy.

Ví dụ

- Cho B-Tree bậc 5 rỗng.
- Hãy xây dựng B-Tree theo thứ tự từ trái sang phải cho dãy số sau:

1	12	8	2	25	5	14	28	17	7	52	16	48	68	3	26	29	53	55	45

Ví dụ

1	12	8	2	25	5	14	28	17	7	52	16	48	68	3	26	29	53	55	45

Chèn 1: | 1 |

Chèn 12: | 1 | 12 |

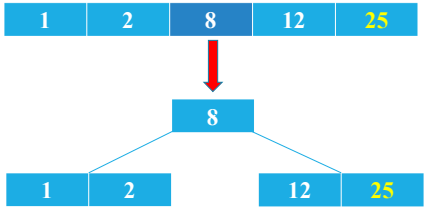
Chèn 8: | 1 | 8 | 12 |

Chèn 2: | 1 | 2 | 8 | 12 |

Ví dụ

1	12	8	2	25	5	14	28	17	7	52	16	48	68	3	26	29	53	55	45

- Do nút gốc đã đầy (4 phần tử) → Chèn 25 vào nút gốc sẽ tách nút gốc thành 2 nút và đưa khóa ở giữa lên trên tạo thành nút gốc mới.



231

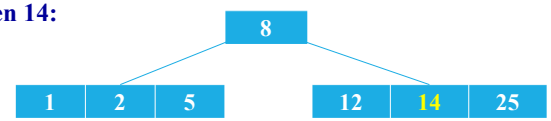
Ví dụ

1	12	8	2	25	5	14	28	17	7	52	16	48	68	3	26	29	53	55	45

- **Chèn 5:**



- **Chèn 14:**

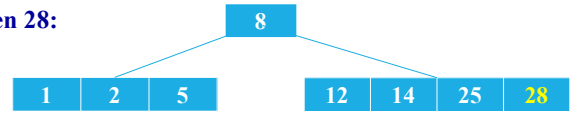


232

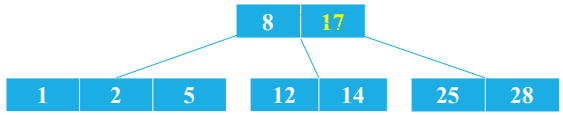
Ví dụ

1	12	8	2	25	5	14	28	17	7	52	16	48	68	3	26	29	53	55	45

- **Chèn 28:**



- **Chèn 17:** do nút lá bên phải đã đầy nên phân đôi và đưa nút giữa lên trên nút cha (nút gốc).

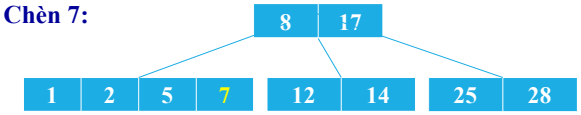


233

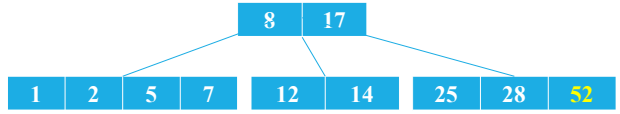
Ví dụ

1	12	8	2	25	5	14	28	17	7	52	16	48	68	3	26	29	53	55	45

- **Chèn 7:**



- **Chèn 52:**

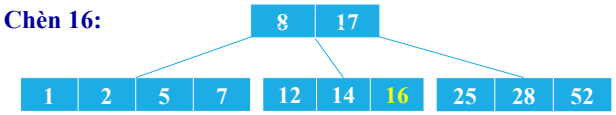


234

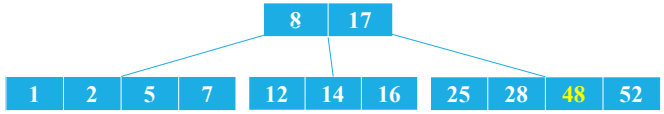
Ví dụ

1	12	8	2	25	5	14	28	17	7	52	16	48	68	3	26	29	53	55	45

❖ **Chèn 16:**



❖ **Chèn 48:**

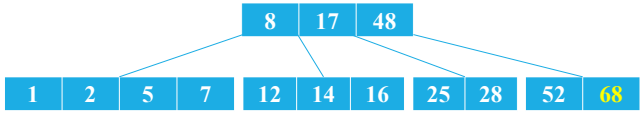


235

Ví dụ

1	12	8	2	25	5	14	28	17	7	52	16	48	68	3	26	29	53	55	45

❖ **Chèn 68:** do nút lá bên phải đã đầy nên phân đôi nút lá và đưa nút giữa lên nút cha (nút gốc).

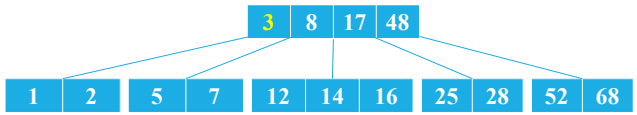


236

Ví dụ

1	12	8	2	25	5	14	28	17	7	52	16	48	68	3	26	29	53	55	45

❖ **Chèn 3:** do nút lá bên phải đã đầy nên phân đôi nút lá và đưa nút giữa lên nút cha (nút gốc).

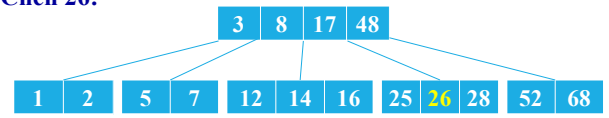


237

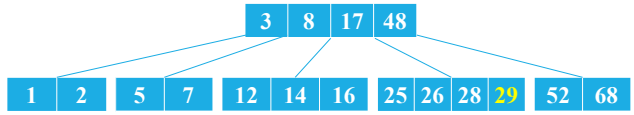
Ví dụ

1	12	8	2	25	5	14	28	17	7	52	16	48	68	3	26	29	53	55	45

❖ **Chèn 26:**



❖ **Chèn 29:**

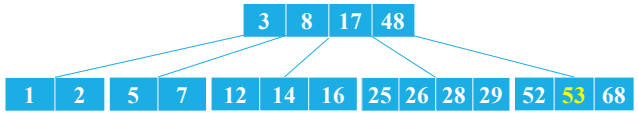


238

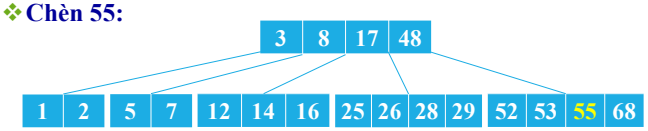
Ví dụ

1 | 12 | 8 | 2 | 25 | 5 | 14 | 28 | 17 | 7 | 52 | 16 | 48 | 68 | 3 | 26 | 29 | 53 | 55 | 45

❖ **Chèn 53:**



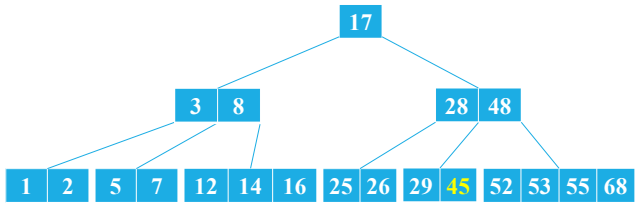
❖ **Chèn 55:**



Ví dụ

1 | 12 | 8 | 2 | 25 | 5 | 14 | 28 | 17 | 7 | 52 | 16 | 48 | 68 | 3 | 26 | 29 | 53 | 55 | 45

❖ **Chèn 45:** do nút lá thứ 3 đầy nên phân đôi và đưa nút giữa lên trên nút cha, nút cha cũng đầy nên phân đôi tiếp nút cha và đưa nút giữa lên trên, tạo thành nút gốc mới.



Xóa khóa trên cây

1. Nếu khóa cần xóa nằm ở nút lá → Xóa.
2. Nếu khóa cần xóa không nằm ở nút lá → Xóa khóa cần xóa, đưa khóa có giá trị gần nhất từ nút lá lên thay thế cho khóa đã xóa.

Xóa khóa trên cây

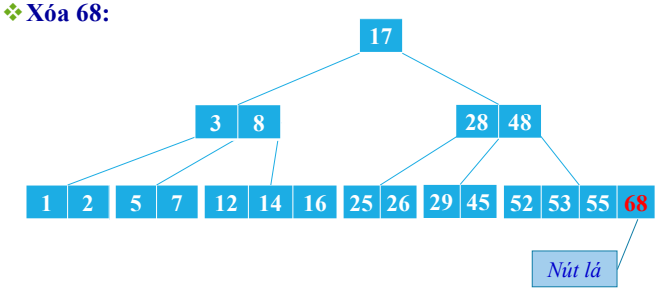
Nếu trong trường hợp (1) hay (2), nút lá còn lại quá ít khóa thì xét các nút anh em kế cận nút đang xét:

3. Nếu một trong các nút anh em kế cận nút đang xét có số lượng khóa nhiều hơn số lượng tối thiểu, đưa một khóa của nút anh em lên nút cha và đưa khóa ở nút cha xuống nút lá có khóa quá ít.
4. Nếu tất cả các nút anh em kế cận nút đang xét đều có số lượng khóa vừa đủ số lượng tối thiểu, chọn một nút anh em kế cận và hợp nhất nút anh em này với nút lá có khóa quá ít.

➔ *Nếu nút cha trở nên thiếu khóa, lặp lại quá trình này.*

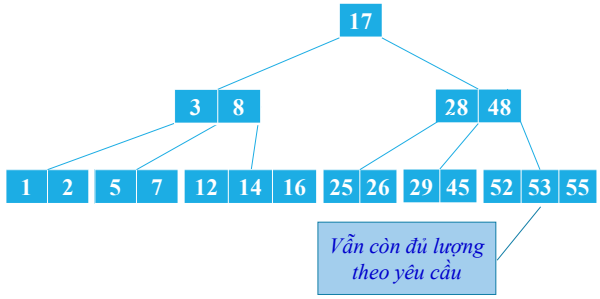
Xóa khóa trên cây

❖ Xóa 68:



244

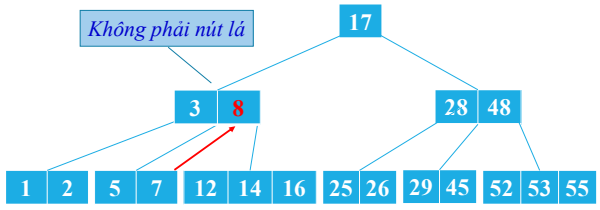
Xóa khóa trên cây



245

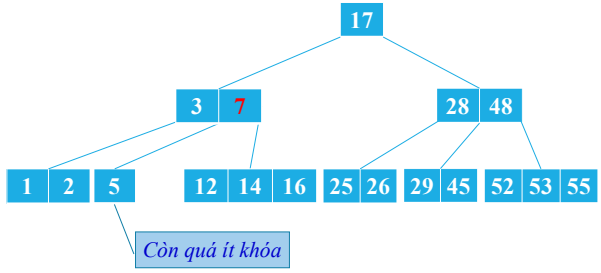
Xóa khóa trên cây

❖ Xóa 8:



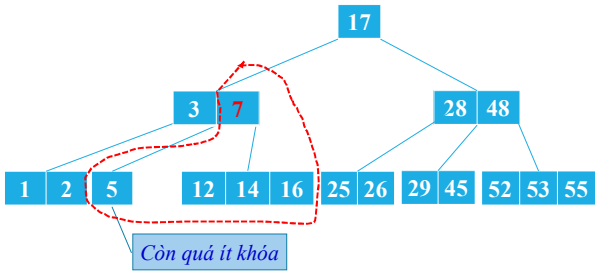
246

Xóa khóa trên cây



247

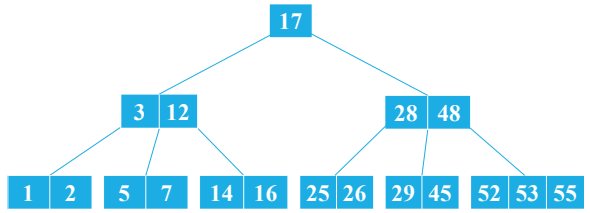
Xóa khóa trên cây



248

Xóa khóa trên cây

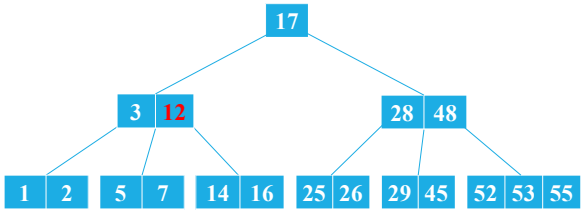
Trường hợp 3: Cân bằng các khóa giữa các nút anh em



249

Xóa khóa trên cây

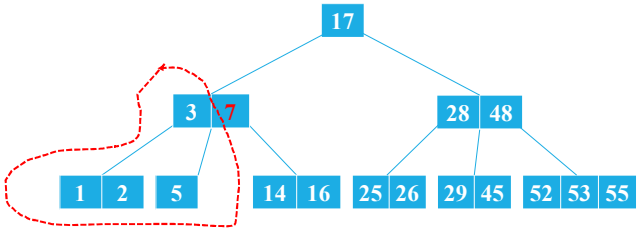
❖ **Xóa 12:**



250

Xóa khóa trên cây

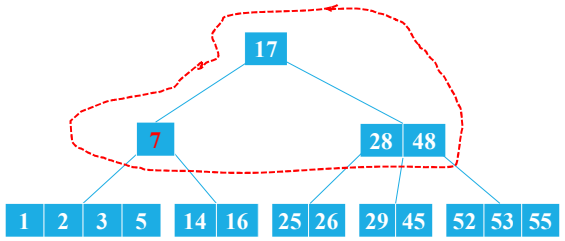
❖ **Xóa 12:**



251

Xóa khóa trên cây

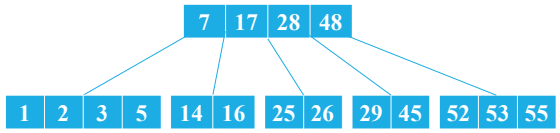
- Xóa 12:



252

Xóa khóa trên cây

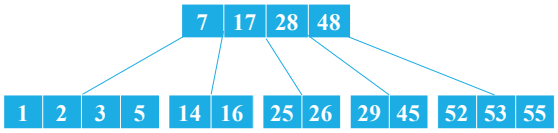
- Xóa 12:



253

Xóa khóa trên cây

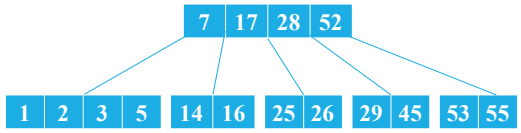
- Xóa 48:



254

Xóa khóa trên cây

- Xóa 48:



255

Cơ sở để so sánh	Cây B	Cây nhị phân
Hạn chế thiết yếu	Một nút có thể có tối đa số M của các nút con (trong đó M là thứ tự của cây).	Một nút có thể có tối đa 2 số cây con.
Đã sử dụng	Nó được sử dụng khi dữ liệu được lưu trữ trên thiết bị ngoại vi (đĩa).	Nó được sử dụng khi các bản ghi và dữ liệu được lưu trữ trong RAM.
Chiều cao của cây	$\log_M N$ (trong đó **M** là thứ tự của cây M-way, **N** là số nút)	$\log_2 N$
Ứng dụng	Mã cấu trúc dữ liệu lập chỉ mục trong nhiều DBMS.	Tối ưu hóa mã, mã hóa Huffman, v.v.

